

## GitHub Task: Lexical Analyzer & Token Counter

### Objective:

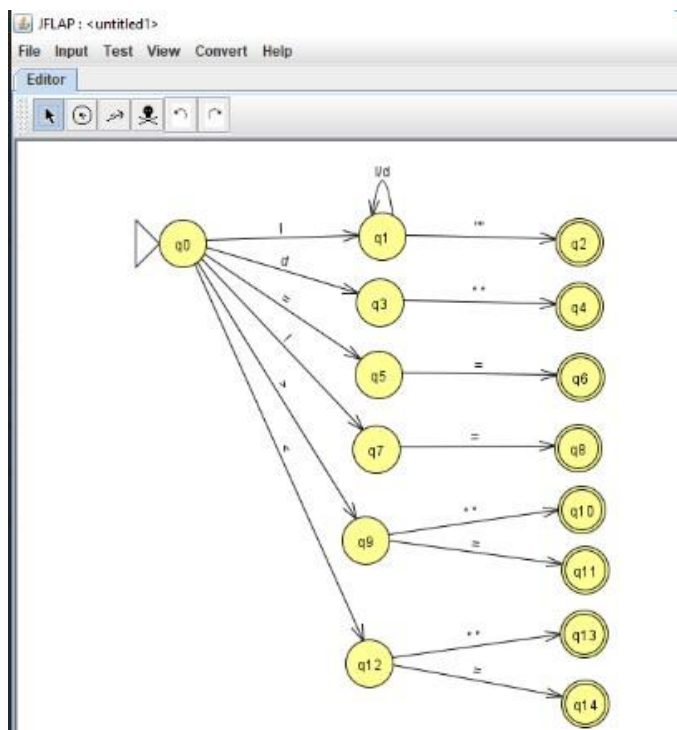
Develop a program that reads a source-code file and performs lexical analysis by identifying and counting different types of tokens.

### Problem Statement:

Implement a program in Python/C/C++/Java that identifies:

- Keywords
- Identifiers
- Operators
- Constants/Literals
- Separators/Delimiters
- Special symbols
- Comments

### Lexical Analyser Transition diagram:



**Source Code:**

```
import java.io.*;

import java.util.*;

public class LexicalAnalyzer {

    static String[] keywords = {

        "auto", "break", "case", "char", "const", "continue",

        "default", "do", "double", "else", "enum", "extern",

        "float", "for", "goto", "if", "int", "long",

        "register", "return", "short", "signed", "sizeof",

        "static", "struct", "switch", "typedef", "union",

        "unsigned", "void", "volatile", "while"

    };

    static boolean isKeyword(String word) {

        for (String keyword : keywords) {

            if (keyword.equals(word))

                return true;

        }

        return false;

    }

    static boolean isSeparator(char ch) {

        return ch == '(' || ch == ')' ||

            ch == '{' || ch == '}' ||

            ch == '[' || ch == ']' ||

            ch == ';' || ch == ',';

    }

    static boolean isOperator(char ch) {

        return ch == '+' || ch == '-' ||

            ch == '*' || ch == '/' ||

            ch == '%' || ch == '=' ||

            ch == '<' || ch == '>' ||

            ch == '!' || ch == '&' ||
```

```
        ch == '|' || ch == '^';
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the source file name: ");
        String filename = scanner.nextLine();
        int keywordCount = 0;
        int identifierCount = 0;
        int operatorCount = 0;
        int constantCount = 0;
        int stringCount = 0;
        int separatorCount = 0;
        int specialCount = 0;
        int commentCount = 0;
        try {
            BufferedReader br = new BufferedReader(new FileReader(filename));
            StringBuilder source = new StringBuilder();
            String line;
            while ((line = br.readLine()) != null) {
                source.append(line).append('\n');
            }
            br.close();
            String code = source.toString();
            int i = 0;
            System.out.println("\nTOKEN\t\tTYPE");
            System.out.println("-----");
            while (i < code.length()) {
                char ch = code.charAt(i);
                if (Character.isWhitespace(ch)) {
                    i++;
                    continue;
                }
            }
        }
    }
}
```

```
}  
  
// Single-line comment  
if (ch == '/' && i + 1 < code.length()  
    && code.charAt(i + 1) == '/') {  
    commentCount++;  
    System.out.println("/\\t\\tComment");  
    i += 2;  
    while (i < code.length() && code.charAt(i) != '\\n')  
        i++;  
    continue;  
}  
  
// Multi-line comment  
if (ch == '/' && i + 1 < code.length()  
    && code.charAt(i + 1) == '*') {  
    commentCount++;  
    System.out.println("/ * *\\t\\tComment");  
    i += 2;  
    while (i + 1 < code.length()  
        && !(code.charAt(i) == '*'  
            && code.charAt(i + 1) == '/')) {  
        i++;  
    }  
    i += 2;  
    continue;  
}  
  
// Identifier or Keyword  
if (Character.isLetter(ch) || ch == '_') {  
    StringBuilder word = new StringBuilder();  
    while (i < code.length()  
        && (Character.isLetterOrDigit(code.charAt(i))  
            || code.charAt(i) == '_')) {
```

```
        word.append(code.charAt(i));
        i++;
    }
    String token = word.toString();
    if (isKeyword(token)) {
        System.out.println(token + "\t\tKeyword");
        keywordCount++;
    } else {
        System.out.println(token + "\t\tIdentifier");
        identifierCount++;
    }
    continue;
}

// Constant
if (Character.isDigit(ch)) {
    StringBuilder number = new StringBuilder();
    while (i < code.length()
        && (Character.isDigit(code.charAt(i))
        || code.charAt(i) == '.')) {
        number.append(code.charAt(i));
        i++;
    }
    System.out.println(number + "\t\tConstant");
    constantCount++;
    continue;
}

// String literal
if (ch == '"') {
    StringBuilder str = new StringBuilder();
    str.append(ch);
    i++;
```

```
while (i < code.length()) {
    char current = code.charAt(i);
    str.append(current);
    i++;
    if (current == '"')
        break;
}
System.out.println(str + "\tString Literal");
stringCount++;
continue;
}
// Character literal
if (ch == '\\') {
    StringBuilder character = new StringBuilder();
    character.append(ch);
    i++;
    while (i < code.length()) {
        char current = code.charAt(i);
        character.append(current);
        i++;
        if (current == '\\')
            break;
    }
    System.out.println(character + "\tCharacter Literal");
    constantCount++;
    continue;
}
// Separators
if (isSeparator(ch)) {
    System.out.println(ch + "\t\tSeparator");
```

```
        separatorCount++;

        i++;

        continue;
    }

    // Operators
    if (isOperator(ch)) {
        String operator = "" + ch;
        if (i + 1 < code.length()) {
            char next = code.charAt(i + 1);
            if ((ch == '+' && (next == '+' || next == '='))
                || (ch == '-' && (next == '-' || next == '='))
                || (ch == '=' && next == '=')
                || (ch == '!' && next == '=')
                || (ch == '<' && next == '=')
                || (ch == '>' && next == '=')
                || (ch == '&' && next == '&')
                || (ch == '|' && next == '|')
                || (ch == '*' && next == '=')
                || (ch == '/' && next == '=')
                || (ch == '%' && next == '=')) {
                operator += next;
                i++;
            }
        }

        System.out.println(operator + "\t\tOperator");

        operatorCount++;

        i++;

        continue;
    }

    // Special symbol
    System.out.println(ch + "\t\tSpecial Symbol");
```

```
        specialCount++;
        i++;
    }
    System.out.println("-----");
    System.out.println("\nTOKEN COUNT");
    System.out.println("-----");
    System.out.println("Keywords      : " + keywordCount);
    System.out.println("Identifiers   : " + identifierCount);
    System.out.println("Operators     : " + operatorCount);
    System.out.println("Constants     : " + constantCount);
    System.out.println("String Literals : " + stringCount);
    System.out.println("Separators    : " + separatorCount);
    System.out.println("Special Symbols : " + specialCount);
    System.out.println("Comments      : " + commentCount);
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + filename);
} catch (IOException e) {
    System.out.println("Error reading the file.");
}
scanner.close();
}
```

**Sample Input:**

```
int sum = a + b;

float average = sum/
2.0;

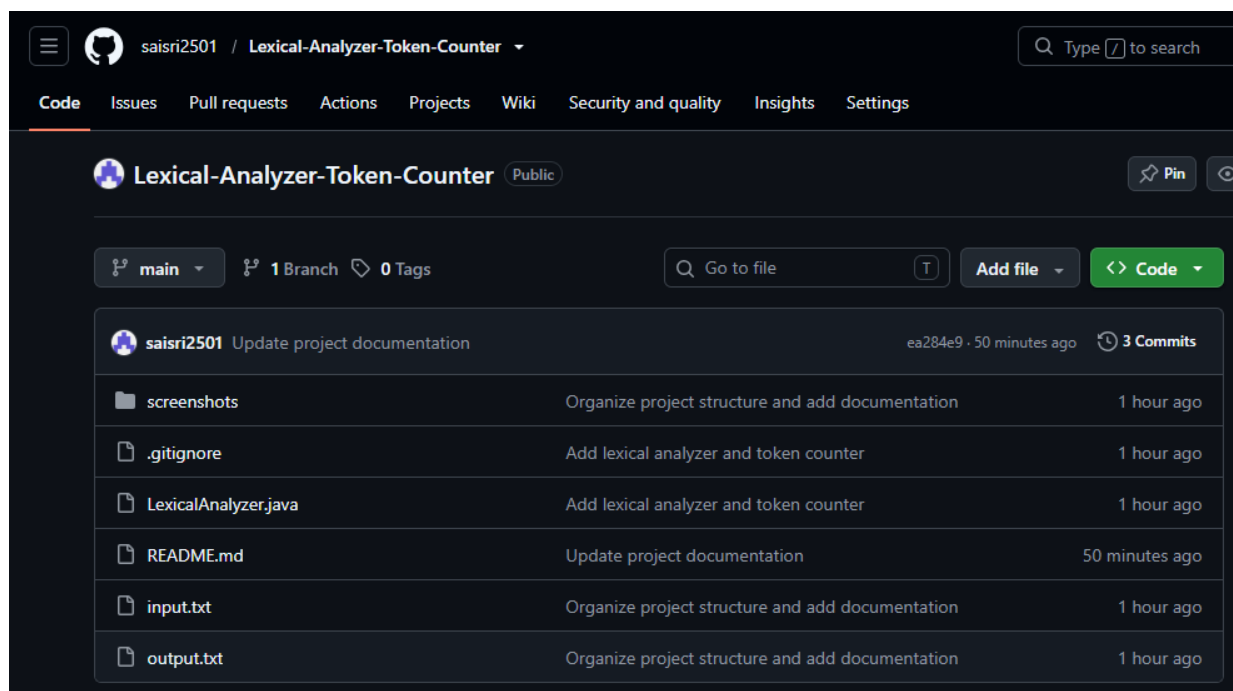
if (average > 50)
    printf("Pass");
```

**Sample Output:****Token Recognition Table:**

| TOKEN   | TYPE           |
|---------|----------------|
| -----   |                |
| int     | Keyword        |
| sum     | Identifier     |
| =       | Operator       |
| a       | Identifier     |
| +       | Operator       |
| b       | Identifier     |
| ;       | Separator      |
| float   | Keyword        |
| average | Identifier     |
| =       | Operator       |
| sum     | Identifier     |
| /       | Operator       |
| 2.0     | Constant       |
| ;       | Separator      |
| //      | Comment        |
| if      | Keyword        |
| (       | Separator      |
| average | Identifier     |
| >       | Operator       |
| 50      | Constant       |
| )       | Separator      |
| printf  | Identifier     |
| (       | Separator      |
| "Pass"  | String Literal |
| )       | Separator      |
| ;       | Separator      |

**Token Count:**

Keywords : 3  
 Identifiers : 7  
 Operators : 5  
 Constants : 2  
 String Literals : 1  
 Separators : 7  
 Special Symbols : 0  
 Comments : 1

**GitHub Repository:****Conclusion:**

The Java-based lexical analyzer successfully reads the C source code from input.txt, identifies and classifies different types of tokens, and counts the occurrences of each token category.

For the given sample input, the program identifies keywords, identifiers, operators, constants, string literals, separators, and comments correctly. The project demonstrates the basic process of lexical analysis used in compiler design